

NCMI - BUCKET-Seq -Team Project - HAHN Erin VAU049
Lab Notes/Caitlin

PDF Version generated by

Caitlin Cherryh

on

Jun 28, 2024 @02:23 PM AEST

Table of Contents

BUCKETseq bioinformatics 2

chromatin-assembly data 7

chromatin-assembly pipeline 9

chromatin-assembly steps 14

chromatin-assembly HPC 15

chromatin-assembly toy pipeline 18

chromatin-assembly running 21

Getting familiar with the new HPC system

1. Create new directory:
 - `mkdir amplicon`
2. Copy file to scratch: samples 31 and 33
 - `cp /datasets/work/efsp-faire/work/vau049/BUCKET-Seq/merged/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq.gz /scratch3/che318/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq.gz`
 - `cp /datasets/work/efsp-faire/work/vau049/BUCKET-Seq/merged/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq.gz /scratch3/che318/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq.gz`
 - `cp /datasets/work/efsp-faire/work/vau049/BUCKET-Seq/merged/amplicon/33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R1.fastq.gz /scratch3/che318/amplicon/33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R1.fastq.gz`
 - `cp /datasets/work/efsp-faire/work/vau049/BUCKET-Seq/merged/amplicon/33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R2.fastq.gz /scratch3/che318/amplicon/33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R2.fastq.gz`
3. Unzip if desired (not required for R scripts or for subsampling):
 - `gzip -d /scratch3/che318/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq.gz`
 - `gzip -d /scratch3/che318/amplicon/31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq.gz`
4. View head and tail of unzipped files
 - `cd /scratch3/che318/amplicon`
 - `(head; tail) < 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq`
 - `(head; tail) < 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq`
5. Subsample file using awk and [this tutorial](#)
 - 153576368 lines in each file!
 - Sampling via awk did not work - code was incorrect and extracting way too many lines (file size > 5GB for k=1, which should extract 1 150bp read).

Fastq subsampling

- Software programs:
 - [SeqKit](#): `./seqkit`
 - [SeqTK](#): `seqtk-master/seqtk`
 - [fastp](#)

Data subsampling

1. Get overview of fastq files with `seqkit stat *.fastq`

file	format	type	num_seqs	sum_len	min_len
avg_len max_len					
31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq	FASTQ	DNA	38,394,092	5,759,113,800	150
150 150					
31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq	FASTQ	DNA	38,394,092	5,759,113,800	150
150 150					

2. Randomly sample 10K reads from the fastq files using Seqkit. Use proportion not number for large fastq files, and specify the seed number for paired end reads to sample the same reads in both files.

```
./seqkit sample -p 0.00026 -s 20240606 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq.gz -o 31_B4_5_12S_sample_R1.fastq.gz
./seqkit sample -p 0.00026 -s 20240606 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq.gz -o 31_B4_5_12S_sample_R2.fastq.gz
./seqkit sample -p 0.00026 -s 20240606 33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R1.fastq.gz -o 33_K_5_12S_sample_R1.fastq.gz
./seqkit sample -p 0.00026 -s 20240606 33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R2.fastq.gz -o 33_K_5_12S_sample_R2.fastq.gz
```

3. Randomly sample 1K reads rom the fastq files using Seqkit, to have a small set to play with

```
./seqkit sample -p 0.000026 -s 20240606 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R1.fastq.gz -o 31_B4_5_12S_sample_small_R1.fastq.gz
./seqkit sample -p 0.000026 -s 20240606 31_B4_5_12S_ATCCGGTA-GACCGATA_L002_R2.fastq.gz -o 31_B4_5_12S_sample_small_R2.fastq.gz
./seqkit sample -p 0.000026 -s 20240606 33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R1.fastq.gz -o 33_K_5_12S_sample_small_R1.fastq.gz
./seqkit sample -p 0.000026 -s 20240606 33_K_5_12S_GTGAAGTG-GAGCAATC_L002_R2.fastq.gz -o 33_K_5_12S_sample_small_R2.fastq.gz
```

Running the existing R scripts

- Running script 01 with small subsampled fastq files
- To get running on server:
 - Want to output plots if possible - investigate graphics device and output
 - Output tracking df and output the input parameters
 - Get running from command line with command line to input filtering variables



[Download](#)

31_B4_5_12S_sample_R1.fastq.gz (401 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

31_B4_5_12S_sample_R2.fastq.gz (450 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

31_B4_5_12S_sample_small_R1.fastq.gz (41.9 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

31_B4_5_12S_sample_small_R2.fastq.gz (47.3 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

33_K_5_12S_sample_R1.fastq.gz (407 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

33_K_5_12S_sample_R2.fastq.gz (445 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

Caitlin Cherryh - Jun 06, 2024, 4:03 PM GMT+10

data



[Download](#)

33_K_5_12S_sample_small_R1.fastq.gz (42.5 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

data



33_K_5_12S_sample_small_R2.fastq.gz (46 kB) Subsampled fastq files from samples 31_B4_5_12S and 33_K_5_12S. Files with "sample" in name have been subsampled to 10K reads, while "samples_small" contain 1K reads.

code



MB00_install_packages.R (728 B) Scripts in R (not R markdown) format to install required packages and perform analysis in R for the first pipeline step MB01.

code



MB01_DADA2.R (7.93 kB) Scripts in R (not R markdown) format to install required packages and perform analysis in R for the first pipeline step MB01.

code

```
# Directory location:
# /usr/src/rdm64/wgfiles/

# Files of interest:
31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R1_Fastq
31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R2_Fastq

# Get overview of files with readseq
readseq view -f Fastq

# Files contain 35,394,092 DNA sequences of 150 bp each.

# To randomly sample 10K reads (proportion of 0.00002) from the Fastq files
readseq sample -p 0.00002 -n 200000000 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R1_Fastq -o 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R1_Sampled_Fastq
readseq sample -p 0.00002 -n 200000000 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R2_Fastq -o 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R2_Sampled_Fastq

# To randomly sample 1K reads from the Fastq files
readseq sample -p 0.00002 -n 2000000 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R1_Fastq -o 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R1_Sampled1K_Fastq
readseq sample -p 0.00002 -n 2000000 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R2_Fastq -o 31_84_5_125_ATCGGCGA_GAGCGATGA_L002_R2_Sampled1K_Fastq
```

subsample_fastq.txt (904 B) Text file with command lines to subsample fastq files on Petrichor.

notes

<https://github.com/caitlinch/BUCKETseq>

Contact Caitlin Cherryh or Makenna Short for access

Issues to resolve

- Script is not outputting graphs when running on server - try:
 - Loading graphviz module (`module load graphviz`)
 - Checking whether specific R packages are required for plotting e.g., `grDevices`



Toy data

Two eastern water dragon (*Intellagama lesueurii lesueurii*) samples mapped against the central bearded dragon (*Pogona vitticeps*) genome.

Reference genome:

- /datasets/work/efsp-faire/work/sti089/genomes/central_dragon

Test sample:

- /datasets/work/efsp-faire/source/ACC_paper/AGRF_CAGRF22019398_H3NTTDSX3/133165_H3NTTDSX3_GTGAAGTG-GAGCAATC_L00*
- The test sample is a female preserved in 2001. The sample was run across 4 lanes resulting in four each forward and reverse read files (total of 8 files).

Input control:

- The appropriate input control for the test sample is a gDNA extract from fresh liver. This sample was run across two flow cells, each with four lanes, resulting in eight each forward and reverse read files (total of 16 files).
- /datasets/work/efsp-faire/source/Tuna_skink_water_dragon/AGRF_CAGRF220510576_HL7J7DSX3/G-WD-1_HL7J7DSX3_ACTTCTGC-CCGGGACT_L00*
- /datasets/work/efsp-faire/source/Tuna_skink_water_dragon/AGRF_CAGRF220510576_HLT7NDSX3/G-WD-1_HLT7NDSX3_ACTTCTGC-CCGGGACT_L00*

To test:

- To check that everything is running, you could start with just one of the R1/R2 read file pairs for the test sample. It's only when you get to the DANPOS3 step that the output is dependent on having the full data set.

File paths from config file:

input_paths:

```
reference_genome_dir: "/datasets/work/efsp-faire/work/sti089/genomes/central_dragon/"
test_sample: "/scratch3/che318/input/ACC_paper/AGRF_CAGRF22019398_H3NTTDSX3/133165_H3NTTDSX3_GTGAAAGTG-
GAGCAATC_L00*"
test_sample_dir: "/scratch3/che318/chromatin-assembly/input/ACC_paper/AGRF_CAGRF22019398_H3NTTDSX3/"
input_control_dir1: "/datasets/work/efsp-faire/source/Tuna_skink_water_dragon/AGRF_CAGRF220510576_HL7J7DSX3/G-WD-
1_HL7J7DSX3_ACTTCTGC-CCGGGACT_L00*"
input_control_dir2: "/datasets/work/efsp-faire/source/Tuna_skink_water_dragon/AGRF_CAGRF220510576_HLT7NDSX3/G-WD-
1_HLT7NDSX3_ACTTCTGC-CCGGGACT_L00*"
```




Pipeline structure

[View pipeline structure in browser](#)

This pipeline makes the following assumptions:

- The `chromatin_assembly_config.yml` file has been updated with slurm parameters for each of the steps
- All files are placed correctly according to the directory structure
- The `<input_file>` for the pipeline run is
`./config/genomes/water_dragons/EWD_hifiasm_HiC.fasta`
- Output log files: `./workflow`

Archival Chromatin Pipeline

Step 1 - Genome preparation

Part 1 - Masking

- **Software:** RepeatMasker
- **Input:**
 1. Fasta file for reference genome, e.g., EWD_hifiasm_HiC.fasta
 2. Illumina adapter file, e.g., adap_cont_rna.fa
- **Output:** Repeat-masked reference genome
- **Example Scripts:**
 1. slurm_waterdragon_RM.sh

Part 2 - Create Suffix Array

- *Create kalign suffix array from the genome*
- **Software:** kalign
- **Input:** repeat-masked reference genome
- **Output:** kalign suffix array
- **Script:**
 - One line in terminal
 - `$ ngskit4b index -i EWD_hifiasm_HiC.fasta.masked -o EWD_hifiasm_HiC.fasta.masked -r waterdragon`

Step 2 - Raw Read QC

- **Software:** FastQC
- **Input:** Paired end read files
- **Output:** FastQC html files
- **Example Scripts:**
 1. slurm_fastqc.sh

Step 3 - UMI Extraction

- **Software:** FGBio, Picard
- **Input:** Raw paired end read files
- **Output:** Paired end read files prepped such that UMIs can be read by PICARD during deduplication in Step 5
- **Example Scripts:**
 1. slurm_extractUMI.sh

Step 4 - Read Alignment

- **Software:** kalign
- **Input:**
 1. Masked genome kalign suffix array from Step 1

2. UMI-extracted paired end reads from Step 3

- **Output:** Alignment file
- **Example Scripts:**
 1. slurm_kalign_umi.sh

Step 5 - Alignment Deduplication

- **Software:** Picard
- **Input:** Alignment file from Step 4
- **Output:** Alignment with PCR & optical duplicates removed
- **Example Scripts:**
 1. slurm_dedup.sh

Step 6 - Alignment GC Bias Correction

- **Software:** deeptools
- **Input:**
 1. Deduplicated alignment file from Step 4
 2. Genome effective size
 3. 2bit genome file
- **Output:** GC bias corrected alignment file
- **Example Scripts:**
 1. slurm_computeGCBias.sh
 2. slurm_correctGCBias.sh
- Note, to calculate effective genome size:
 - Run faCount - https://hgdownload.soe.ucsc.edu/admin/exe/linux.x86_64/
- Note, to generate the 2bit genome:
 - \$ module load ucsc tools
 - \$ faToTwoBit EWD_hifiasm_HiC.fasta.masked EWD_hifiasm_HiC.fasta.masked.2bit

Step 7 - Summary Statistics

- *Calculate alignment stats & estimate mean nuclear genome cover*
- **Software:** Picard
- **Input:** GC-corrected alignment file from Step 6
- **Output:** Size metrics summary files (.txt & .pdf)
- **Example Scripts:**
 1. slurm_sizemetrics.sh
 2. slurm_read_count_GCcorrect.sh
- Note, the second script gives you the number of aligned reads. To compute genome cover, we multiply the number of aligned reads by the mean insert length divided by the unmasked genome size.

Step 8 - Peak Analysis

- **Software:** DANPOS3
- **Input:** GC-corrected alignments for TEST & INPUT samples
- **Output:** Folder with DANPOS results (lots of files)
- **Example Scripts:**
 1. slurm_dtriple_1_vs_27-29.sh
- Note, this script runs sample number 1 against a pool of three samples (27-29) as an input control

Caitlin Cherryh - Jun 14, 2024, 4:34 PM GMT+10

planning

pipeline

Pipeline design requirements:

- Runs on server with options to adjust number of cores, run time, and memory
- Allow individual sections of the pipeline to be run independently
- Pipeline managed by config file
 - End user creates and modifies
 - Does not require end user to be familiar with individual programs included in pipeline
 - Controls pipeline run
 - Contains file paths (data, directories) and options to feed into software during specific pipeline step
- Pipeline can be downloaded and used by collaborators independently
 - Does not need any specific
- Facilitates replicable research
 - Config file sufficient to replicate results
 - Ideally would output the programs used and reminders to cite each program
 - Ideally would output command lines and output from each software as log file

Caitlin Cherryh - Jun 14, 2024, 4:33 PM GMT+10

pipeline

GitHub repository

caitlinch/chromatin-assembly

<https://github.com/caitlinch/chromatin-assembly>

Private repository - contact Caitlin Cherryh for access

To do:

Software required

- faCount (https://hgdownload.soe.ucsc.edu/admin/exe/linux.x86_64/)

Pipeline creation:

- ~~Set up snakemake conda environment~~
- ~~Snakemake familiarisation: see [here](#) for a nice tutorial, [here](#) for the Snakemake documentation tutorial, and [here](#) for a nice tutorial with some formatting ideas~~
- ~~Download required software~~
- ~~Identify HPC modules required~~
- ~~Create config file structure and necessary parameters~~
- ~~Structure directory~~
- ~~Identify necessary rules and create rule outlines~~
- P1a: Genome preparation
- P1b: Suffix array
- P2: Raw read QC
- P3: UMI extraction
- P4: Read alignment
- P5: Alignment deduplication
- P6: Alignment GC bias correction
- P7: Calculate alignment stats and mean nuclear genome coverage
- P8: Peak analysis
- Get Slurm running on HPC
- Test whether Slurm parameters should be in sbatch file or called by Snakemake
- Get pipeline running on Slurm

Potential issues:

- Managing Slurm within Snakemake on the HPC
- Accessing Snakemake
 - Available in the python module (module load python) - see [here](#) and [here](#) for example pipelines on the CSIRO HPC
- Snakemake best practices

Questions/clarifications/issues

1. Why are different versions of Picard are used in different scripts? Is it possible to only use the most recent version in both scripts (could improve reproducibility)?
2. Adapters for RepeatMasker - should I use the "adap_cont_rna.fa"?



Step 1: Mask repeats

Step 1: Mask repeats with RepeatMasker

- RepeatMasker [website](#) and [documentation](#)
- Command line: RepeatMasker -pa <num_cores> -qq -dir <out_dir> -lib resources/adapters/adap_cont_rna.fa <input_fasta>
 - <out_dir> = output directory
 - <input_fasta> = input fasta file
- Command options:
 - -pa N : Number of processors to use in parallel (only works for batch files or sequences larger than 50 kb)
 - -qq : Rush job (about 10% less sensitive)
 - -dir : specify directory
 - -lib : Allows the use of a custom library. Skips most of processing, does not produce a .tbl file unless the custom library is in the >name#class format.
- Input: Reference genome file
 - fasta = config["input"]["reference_genome_dir"] + config["input"]["reference_genome_file"]
- Output: masked output file
 - masked = OUTDIR + config["input"]["reference_genome_file"] + ".masked"

Step 2: Quality control with FastQC

- Input:
 - Pair of reads (*_R1.fastq.gz, *_R2.fastq.gz)
 - r1 = config["input_dir"] + SAMPLE_NAME + "{num}_R1.fastq.gz"
 - r2 = config["input_dir"] + SAMPLE_NAME + "{num}_R2.fastq.gz"
- Output:
 - Zip and HTML files for one set of reads
 - Command should generate files for both, but only checks for one set (checks for R1)
 - html = OUTDIR + "fastqc/" + SAMPLE_NAME + "{num}_R1_fastq.html"
 - zip = OUTDIR + "fastqc/" + SAMPLE_NAME + "{num}_R1_fastq.zip"
- Command options:
 - -t = number of files to process simultaneously ([see doco](#))

Step 3: UMI extraction with FGBio and Picard

- Input:
 - Raw paired end read files
- Output:
 - Paired end read files prepped such that UMIs can be read by PICARD during deduplication in Step 5
- Command options:
 -



Virtual environments

- Accessing conda
 - Load the miniconda3 module: `module load miniconda3`
 - If shell is not initialised: `conda init bash`
 - If need to install conda packages: `source /apps/miniconda3/enable_miniconda.sh`
- Accessing virtualenv
 - Load the python module: `module load python`
 - If virtualenv not available, run: `source $(which virtualenvwrapper_lazy.sh)`
- Activating different virtual environments:
 - To activate virtualenv: `workon snakemake`
 - To activate environment: `conda activate snakemake`
- I have created a conda environment called `chromatin-assembly` for developing this pipeline, containing the modules listed below

Modules

- miniconda3 - create conda environments (miniconda3)
- python - contains Snakemake (see [here](#)) (python)
- Bioinformatics:
 - [RepeatMasker](#) (repeatmasker/4.1.0) + [doco](#)
 - [kalign](#) (kit4b/200218) + [index command doco](#)
 - FastQC (fastqc/0.11.9)
 - FGBio (fgbio/1.3.0)
 - Picard (picard/2.26.10, picard/2.26.9)
 - deeptools (samtools/1.12)
 - DANPOS3 (danpos/3.1.1)
 - faCount downloaded from https://hgdownload.soe.ucsc.edu/admin/exe/linux.x86_64/ (path: /scratch3/che318/chromatin-assembly/faCount)

To load modules: `load module <module>` or `load module <module>/<version>`

To list open modules: `module list`

To output list of open modules: `module list > modules.txt`

Snakemake

Command line options

- `-n`: Used in command `snakemake -n` to perform dry run. Useful to test if the workflow is defined properly and to estimate the amount of needed computation.
- `--reason, -r`: Print the reason for each executed rule (deprecated, always true now).
- `--printshellcmds, -p`: Print out the shell commands that will be executed. (default: False)
- `--cores N, -cN`: Specify N cores (necessary on HPC)
- `--summary, -S`: Print a summary of all files created by the workflow. The has the following columns: filename, modification time, rule version, status, plan. Thereby rule version contains the version the file was created with (see the version keyword of rules), and status denotes whether the file is missing, its input files are newer or if version or implementation of the rule changed since file creation. Finally the last column denotes whether the file will be updated or created during the next workflow execution. (default: False)

Command lines

- **Running snakemake pipelines with 1 core (-c1):**
 - To dry run:
 - `snakemake --dryrun --reason --printshellcmds <output_file> -c1`
 - `snamemake -n -r <output_file> -c1`
 - To run:
 - `snakemake --reason --printshellcmds <output_file> -c1`
 - `snakemake -r <output_file> -c1`
 - `<output_file>` should be the desired output file. Snakemake will work backwards by identifying which commands must be run to generate that output file.
 - To specify a Snakefile:
 - `snakefile -s path/to/Snakefile -n -r <output_file> -c1`
- **Output records of pipeline runs:**
 - `snakemake <output_file> -D > summary.tsv`
 - Snakemake keeps track of how all files were generated (when, by which rule, which version of the rule, and by which commands). You can export this information to a tab-delimited file.
 - Snakemake doesn't regenerate files by default if the rule implementation changes.
 - Important to look at the summary table and rerun things as needed.
 - You can get a list of the files for which the rule implementation has changed, and then force Snakemake to regenerate these files with the `-R` flag
- **Rule graphs** show how the rules are connected
 - To create graph of rules: `snakemake --rulegraph <output_file>| dot -Tpng > rulegraph.png`
 - To output png files, load the GraphViz module first: `module load graphviz`
 - `<output_file>` here should be the desired output file from running the snakemake pipeline e.g. for [the example in the tutorial](#), `<output_file>` is `a_b.txt`
- **Jobs graphs** show how the jobs (i.e., an execution of a rule with some given inputs/outputs/settings) are connected
 - To create graph of jobs: `snakemake --dag <output_file>| dot -Tpng > daggraph.png`

- To output png files, load the GraphViz module first: `module load graphviz`
- `<output_file>` here should be the desired output file from running the snakemake pipeline
e.g. for [the example in the tutorial](#), `<output_file>` is `a_b.txt`

Using environment modules

Snakemake allows to define environment modules per rule ([see documentation](#)), listed within the rule like so:

```
envmodules:  
  "bio/bwa/0.7.9",  
  "bio/samtools/1.9"
```

- To execute on HPC: `snakemake --use-envmodules`
- To run outside the HPC system where the workflow was designed:
`snakemake --software-deployment-method conda`

Examples

- Example Snakemake yaml config file: [config/config.yaml](#)
- Example Snakemake workflow: [dna-seq-gatk-variant-calling](#)

Caitlin Cherryh - Jun 14, 2024, 4:27 PM GMT+10

[notes](#) [resources](#)

Tools

- To generate lovely repository tree diagrams: <https://tree.nathanfriend.io/>

Caitlin Cherryh - Jun 27, 2024, 4:19 PM GMT+10

Project code

Please use the project code OD-233464 for all your jobs. It's just a way of tracking which projects are using the HPC resources.

```
#SBATCH --account=OD-233464
```

Or you can set it when you submit the job. I tend to do it this way as I use various project codes.

```
$ sbatch -A OD-233464 slurm_script.sh
```


Toy Snakemake pipeline

Directory structure

```

.
└─ che318/
    └─ snakemake_tut/ **RUN PIPELINE HERE **
        └─ workflow/
            └─ config.yaml
            └─ Snakefile
            └─ results/
                └─ a.upper.txt
                └─ b.upper.txt
                └─ a_b.txt
        └─ test_input/
            └─ a.txt
            └─ b.txt

```

Config file: /scratch/che318/snakemake_tut/workflow/config.yaml

```

input:
  dir: "/scratch3/che318/test_input/"
output:
  dir: "/scratch3/che318/snakemake_tut/results/"

```

Snakefile: /scratch/che318/snakemake_tut/workflow/config.yaml

```

configfile: "workflow/config.yaml"

```

```

rule convert_to_upper_case:
  input:
    file = config["input"]["dir"] + "{name}.txt"
  output:
    file = config["output"]["dir"] + "{name}.upper.txt"
  shell:
    "tr [a-z] [A-Z] < {input.file} > {output.file}"

rule concatenate_files:
  input:
    file1 = config["output"]["dir"] + "{first}.upper.txt",
    file2 = config["output"]["dir"] + "{second}.upper.txt"
  output:
    config["output"]["dir"] + "{first}_{second}.txt"
  shell:
    "echo 'Concatenating {input}' | cat - {input.file1} {input.file2} > {output}"

```

Running toy pipeline: /scratch/che318/snakemake_tut/

PREP:

```

module load python

```

```
cd /scratch3/che318/snakemake_tut/
```

DRY RUN:

```
snakemake -s workflow/Snakefile --dryrun --reason --printshellcmds  
/scratch3/che318/snakemake_tut/results/a_b.txt -c1
```

FULL RUN:

```
snakemake -s workflow/Snakefile --reason --printshellcmds  
/scratch3/che318/snakemake_tut/results/a_b.txt -c1
```

GENERALISED EXAMPLES

DRY RUN: `snakemake -s relative/path/to/Snakefile -n -r /full/path/to/output/file.txt -c1`

RUN: `snakemake -s relative/path/to/Snakefile -r /full/path/to/output/file.txt -c1`

Remember:

- Move to the directory containing the directory for the pipeline, i.e., `scratch3/che318/`
- Specify **full path** to any files within the config file `config.yaml`
 - CORRECT: `/scratch3/che318/test_input/a.txt`
- Specify **full path** to all output files in command line (as Snakemake will define output files using the full output path given within the config file)
 - INCORRECT: `results/a_b.txt`
 - CORRECT: `/scratch3/che318/snakemake_tut/results/a_b.txt`
- Specify **relative path** to config file within Snakefile:
 - Assuming you are running the pipeline from within the directory `/scratch3/che318/`:
 - `configfile: "snakemake_tut/workflow/config.yaml"`
 - This way, the path to the configfile does not need to be updated when the Snakefile is moved around to different computers

